LES BASES DE LA PROGRAMMATION

Driss MATROUF
Professeur des universités
Avignon Université

4 crédits, 1 examen écrit (50%), un. Examen TP (50%)

Objectifs du cours

- Comprendre comment un programme est exécuté par une machine.
- Écrire les premiers programmes Python : variables, types, entrées/sorties.
- Utiliser les tests, les booléens et les boucles pour contrôler l'exécution.
- Manipuler des listes : création, accès, modification, parcours.
- Savoir lire et corriger un code Python en respectant strictement l'indentation.

Structure générale d’un ordinateur



CPU

Central Processing Unit : exécute les instructions.

Mémoire

ROM non volatile, RAM volatile.

Périphériques E/S

Disque, clavier, écran, lecteur, graveur…

Exécution d’un programme

Le CPU répète un cycle instruction par instruction

1

Chercher

une instruction dans la RAM

2

Exécuter

l’opération demandée

3

Avancer

vers l’instruction suivante

Une instruction = code de l’opération + opérandes

Exemples : addition, multiplication, comparaison, affectation…

Les différents niveaux de langage

Langage machine

Propre à chaque processeur. Directement exécutable par la machine. Non portable.

Assembleur

Codage alphanumérique du langage machine. Traduit par un assembleur. Toujours peu portable.

Haut niveau

Normalisé, plus lisible et portable. Traduit par un compilateur ou un interpréteur : C, Pascal, C++, Java, Python…

Production des programmes

Compilation

Traduction du source en langage objet, puis édition de liens pour produire un exécutable.



La compilation réalise plusieurs analyses : lexicale, syntaxique et sémantique.

Interprétation

Traduction et exécution au fur et à mesure des instructions.



L'interprétation facilite le développement, mais peut introduire une certaine lenteur.

Technique de production de Python

Python utilise une technique mixte : compilation en bytecode, puis interprétation du bytecode.



Portabilité

Le bytecode peut être exécuté sur toute machine disposant d'une machine virtuelle Python.

Précompilation

Python peut réutiliser des versions précompilées des modules : .pyc.

Python : caractéristiques importantes

- Langage sorti en 1991, associé à la Python Software Foundation depuis 2001.
- Langage puissant, facile à apprendre et riche en possibilités.
- Langage interprété, open source et orienté objet.
- Travail interactif et documentation en ligne importante.
- Indentation significative : l'indentation fait partie de la syntaxe.
- Grand écosystème de bibliothèques et de packages.

Algorithme, programme, module

Algorithme

Suite d'étapes permettant d'atteindre un but donné.

Programme

Traduction d'un algorithme dans un langage compilable ou interprétable.

Module

Programme pouvant être importé pour réutiliser des fonctions.

```
import math          # importe le module complet
from math import cos # importe une fonction précise
```

Premiers pas avec l’interpréteur

```
$python3
>>> 3 + 7
10
>>> bonjour
Traceback ...
NameError: name 'bonjour' is not defined
```

Idée importante

L’interpréteur n’est pas une conversation : il attend des expressions ou des instructions Python.

Chaînes de caractères

Pour afficher du texte, on écrit par exemple : print("bonjour").

Les opérations et les nombres

```
>>> 3 + 7
10
>>> 3.0 + 7
10.0
>>> 10 / 3
3.3333333333333335
>>> 13 // 3
4
>>> 13 % 3
1
```

/

division réelle

//

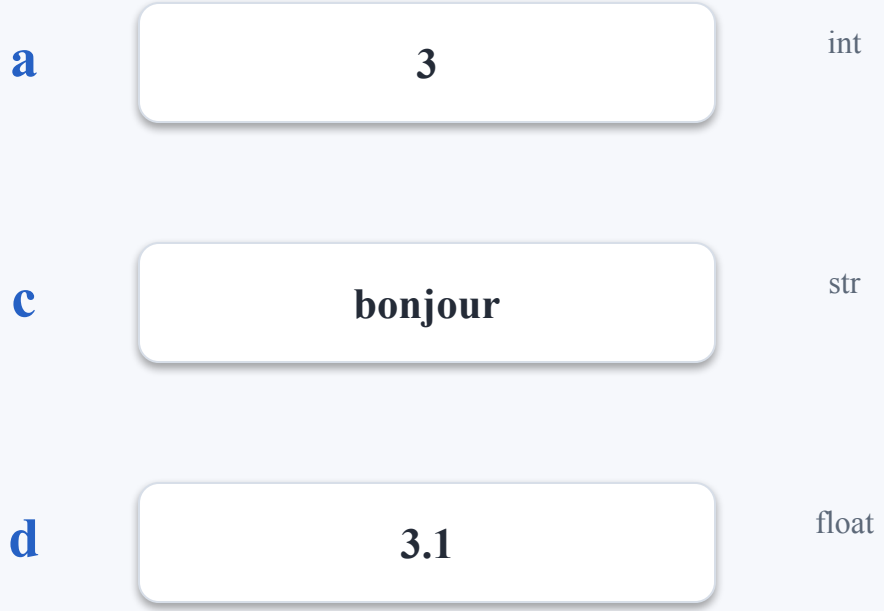
division entière

%

reste de la division : modulo

Variable : nommer une valeur

```
a = 3
c = "bonjour"
d = 3.1
```



Une variable est un nom qui permet de retrouver et de manipuler une valeur.

Variables : noms et mots réservés

```
damien_age = 21
print(damien_age)

damien_age = damien_age + 2
print(damien_age)
```

Noms explicites

Préférer damien_age à x quand le sens est important.

Mots réservés

Ne pas utiliser if, while, for, class, return… comme noms de variables.

Convention courante en Python : noms en minuscules avec des underscores.

Les types des variables

```
a = 3          # int
a = 3.1        # float
a = "bonjour"  # str

print(type(a))
```

int

entiers : 3, 0, -12

float

nombres réels : 3.1, -0.5

str

chaînes de caractères : "bonjour"

Le type booléen

Un booléen ne peut prendre que deux valeurs : True ou False.

```
a = 2 > 3
print(a)        # False

damien_age = 20
b = damien_age < 30
print(b)        # True
```

Comparaisons

==, !=, >, >=, <, <=

Attention

True et False commencent par une majuscule.

Expressions booléennes

- Une expression logique produit un résultat vrai ou faux.
- Elle est utilisée dans les conditions et les boucles.
- Opérateurs relationnels : >, <, ==, !=, >=, <=.
- Opérateurs booléens : and, or, not.
- Python peut arrêter l'évaluation dès que le résultat est déterminé.

print : afficher des résultats

```
print(5)

a = 8
b = 2

print(a)
print(a, b)
print(a, "puis", b)
```

Sortie

print affiche dans la console.

Python 3

Les parenthèses sont obligatoires.

Plusieurs valeurs

print(a, b) ajoute un espace automatiquement.

input : saisir au clavier

```
a = input()                    # str

a = int(input())               # entier
x = float(input())             # réel

a = int(input("Entrez une valeur entière : "))
```

Règle

input retourne toujours une chaîne de caractères.

Conversion

int(...) et float(...) convertissent la saisie.

Instruction conditionnelle : if

```
if expression_logique:
    commandes

if expression_logique:
    commandes1
else:
    commandes2

if expression1:
    commandes1
elif expression2:
    commandes2
else:
    commandes3
```

Deux-points

La ligne if se termine par :

Indentation

Le bloc exécuté est indenté.

else / elif

Ils sont alignés avec le if correspondant.

Exemple : if

```
a = int(input("Entrez une valeur : "))

if a < 0:
    print(a, "est négatif")

print("Merci")
```

Si a = 2

Le test est faux : seul Merci est affiché.

Si a = -2

Le test est vrai : on affiche aussi “-2 est négatif”.

if … else

```
a = int(input("Entrer une valeur : "))

if a < 0:
    print(a, "est négatif")
else:
    print(a, "est positif ou nul")

print("Merci")
```

Branche if

Exécutée si a < 0.

Branche else

Exécutée dans tous les autres cas.

if … elif … else

```
a = int(input("Entrer une valeur : "))

if a < 0:
    print(a, "est négatif")
elif a == 0:
    print(a, "est nul")
else:
    print(a, "est positif")
```

a < 0

négatif

a == 0

nul

sinon

positif

Retour aux booléens

Programme : saisir une valeur entre 10 et 20, la multiplier par 2, afficher le résultat.

```
x = int(input("Entrez une valeur comprise entre 10 et 20 : "))

if x >= 10 and x <= 20:
    print("Le résultat est :", x * 2)
else:
    print("La valeur doit être comprise entre 10 et 20")

print("Au revoir")
```

Test booléen : multiple

Programme : demander deux entiers et dire si l'un est multiple de l'autre.

```
B = False
x = int(input("x = "))
y = int(input("y = "))

if x >= y:
    if x % y == 0:
        B = True
else:
    if y % x == 0:
        B = True

if B:
    print("OUI")
else:
    print("NON")
```

Version corrigée : les mots-clés Python sont en minuscules : if, else. Les blocs imbriqués ont une indentation supplémentaire.

Les boucles

Principe

L'ordinateur lit les instructions de haut en bas, puis repart au début de la boucle tant que nécessaire.

Deux familles

while : condition de continuation.
for : parcours d'une séquence ou d'un intervalle.

Une boucle sert à répéter un bloc d'instructions. Il faut toujours répondre à 3 questions :
1- où commence-t-on ?
2- jusqu'à quand continue-t-on ?
3- qu'est qui change à chaque tour (ou à chaque itération)

Une boucle doit toujours avoir une condition d'arrêt ou une progression.

while

```
while expression_logique:
    instruction1
    instruction2

# Exemple
i = 0
while i < 6:
    print("Bonjour", i)
    i = i + 1
```

Condition

La boucle continue tant que la condition est vraie.

Progression

i = i + 1 rapproche la boucle de sa fin.

while

Tour	i avant le test	i < 6	affichage	i après
1	0	True	Bonjour 0	1
2	1	True	Bonjour 1	2
3	2	True	Bonjour 2	3
4	3	True	Bonjour 3	4
5	4	True	Bonjour 4	5
6	5	True	Bonjour 5	6
-	6	False	-	-

Boucle infinie

```
i = 0
while i < 6:
    print("Bonjour", i)

# i ne change jamais :
# la condition reste vraie
```

Résultat

```
Bonjour 0
Bonjour 0
Bonjour 0
Bonjour 0
...
```

Erreur fréquente : oublier de modifier la variable testée dans la condition.

Exo 1 : table de multiplication

Demander un entier entre 2 et 9 et afficher sa table de multiplication.

```
n = int(input("Entrez un entier compris entre 2 et 9 : "))

while n < 2 or n > 9:
    n = int(input("Entrez un entier compris entre 2 et 9 : "))

i = 1
while i < 10:
    print(i, "x", n, "=", i * n)
    i = i + 1
```

Encore while : calculer 2^n

Programme : demander un entier positif n et afficher 2^n.

```
n = int(input("Entrez un entier > 0 : "))

while n <= 0:
    n = int(input("Entrez un entier > 0 : "))

i = 0
P = 1
while i < n:
    P = P * 2
    i = i + 1

print("Le résultat est :", P)
```

Ajout utile : on corrige la syntaxe Python. Il faut un deux-points après while i < n et écrire i = i + 1.

Boucles imbriquées : carré d’étoiles

```
n = int(input("Entrer un entier : "))

i = 0
while i < n:
    j = 0
    while j < n:
        print("* ", end="")
        j = j + 1
    print("\n", end="")
    i = i + 1
```

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Chaque niveau de boucle ajoute un niveau d’indentation.

Devinez, je te guide !

```
Secret = 7
nombre = int(input("Devinez le nombre :"))
while nombre != secret:
    if nombre < secret:
        print("Trop petit")
    else:
        peint("Trop grand")

    nombre = int(input("Essayez encore:"))

Print("Bravo, vous avez trouvé!")
```

```
Devinez le nombre:8
Trop grand
Devinez le nombre:5
Trop petit
Devinez le nombre:7
Bravo, vous avez trouvé
```

Chaque niveau de boucle ajoute un niveau d'indentation.

Boucles avec for

```
for i in H:
    traiter(i)

# H peut être une liste
for i in [1, 2, 3, 4, 5]:
    print(i)

# H peut être un range
for i in range(1, 6):
    print(i)
```

for

Parcourt les éléments d’une collection.

range

Produit une suite d’entiers.

FOR : comprendre range

```
for i in range(7):
    print(i)
# 0 1 2 3 4 5 6

for i in range(0, 10, 2):
    print(i)
# 0 2 4 6 8
```

range(7)

va de 0 à 6

range(0, 10, 2)

va de 0 à 9 avec un pas de 2

La borne de fin est exclue.

for à la place du while

Même exercice : table de multiplication, mais la répétition est écrite avec for.

```
n = int(input("Entrez un entier compris entre 2 et 9 : "))

while n < 2 or n > 9:
   n = int(input("Entrez un entier compris entre 2 et 9 : "))

for i in range(1, 10):
   print(i, "x", n, "=", i * n)
```

Avec for, il ne faut pas écrire i = i + 1 : range fournit automatiquement les valeurs de i.

La moyenne de valeurs saisies

La saisie se termine quand l'utilisateur saisit une valeur négative.

```
x = float(input("Entrez une valeur : "))
somme = 0.0
compteur = 0

while x >= 0:
     somme = somme + x
     compteur = compteur + 1
     x = float(input("Entrez une valeur : "))

if compteur > 0:
     moyenne = somme / compteur
     print("La moyenne est :", moyenne)
else:
     print("La moyenne n'est pas calculée")
```

La moyenne : voulez-vous continuer ?

La saisie s'arrête quand l'utilisateur répond non à la question "D'autres valeurs ?".

```
somme = 0.0
compteur = 0
continuer = True

while continuer:
     x = float(input("Entrez une valeur : "))
     somme = somme + x
     compteur = compteur + 1
     rep = input("D'autres valeurs ? ")
     if rep == "non":
          continuer = False

if compteur > 0:
     moyenne = somme / compteur
     print("La moyenne est :", moyenne)
else:
     print("le moyenne ne peut pas être calculée")
```

Correction importante : en Python, on écrit False avec une majuscule.

break dans une boucle

```
somme = 0.0
compteur = 0

while True:
    x = float(input("Entrez une valeur : "))
    somme = somme + x
    compteur = compteur + 1
    rep = input("D'autres valeurs ? ")
    if rep == "non":
                break

if compteur > 0:
    moyenne = somme / compteur
    print("La moyenne est :", moyenne)
```

break

Sort immédiatement de la boucle.

while True

Boucle volontairement infinie, arrêtée par break.

Les listes

- Une liste est un objet pouvant contenir d'autres objets.
- Les objets contenus peuvent être de types différents : int, float, str, listes…
- L'implémentation interne est cachée : on utilise l'objet via ses opérations.
- Opérations principales : création, parcours, ajout, suppression.

```
ma_liste = list()
print(type(ma_liste))  # <class 'list'>
```

Créer une liste

```
ma_liste = []

ma_liste = [1, 2, 3, 4, 5]
print(ma_liste)
# [1, 2, 3, 4, 5]

ma_liste = [1, 3.5, "une chaine", []]
```

Liste vide

[] ou list()

Hétérogène

Une liste peut contenir des types différents.

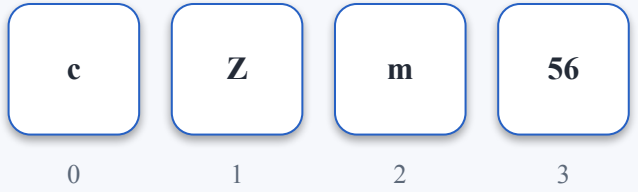
Accéder aux éléments d’une liste

```
ma_liste = ['c', 'f', 'm']

print(ma_liste[0])  # premier élément : 'c'
print(ma_liste[2])  # troisième élément : 'm'

ma_liste[1] = 'Z'
print(ma_liste)     # ['c', 'Z', 'm']

ma_liste.append(56)
print(ma_liste)     # ['c', 'Z', 'm', 56]
```



Les indices commencent à 0.

Attention : une liste n’est pas un tableau C/C++

Tableau C/C++

- Suite contiguë d’octets en mémoire. Taille et type souvent très contraints.
- Le programme demande au système d’exploitation de lui allouer n octets
- C’est le programme qui doit demander au système d’exploitation de reprendre la mémoire allouée (libérer)

Liste Python

- Objet de plus haut niveau. On n’a pas besoin de connaître son implémentation pour l’utiliser.
- C’est python qui s’occupe de libérer la mémoire grâce au **comptage de références**
- a = [1,2,3]
  b = a #dans ce cas le compteur = 2

Pour apprendre Python, on raisonne d’abord avec les opérations de liste, pas avec la représentation mémoire.

Les listes sont modifiables

```
liste1 = [1, 5.5, 18]
liste2 = liste1.append(-15)

print(liste1)  # [1, 5.5, 18, -15]
print(liste2)  # None
```

Mutable

append modifie la liste elle-même.

Retour

La méthode append ne renvoie pas une nouvelle liste : elle renvoie None.

Ajouter et concaténer des listes

```
ma_liste = ['a', 'b', 'd', 'e']
ma_liste.insert(2, 'c')
print(ma_liste)
# ['a', 'b', 'c', 'd', 'e']

ma_liste1 = [3, 4, 5]
ma_liste2 = [8, 9, 10]
ma_liste1.extend(ma_liste2)
print(ma_liste1)
# [3, 4, 5, 8, 9, 10]
```

insert(i, x)

insère x à l'indice i.

extend(L2)

ajoute les éléments de L2 à la fin.

Suppression d’un élément d’une liste

```
ma_liste = [-5, -2, 1, 4, 7, 10]

del ma_liste[0]
print(ma_liste)   # [-2, 1, 4, 7, 10]

del ma_liste[2]
print(ma_liste)   # [-2, 1, 7, 10]

ma_liste = [31, 32, 33, 34, 35]
ma_liste.remove(32)
print(ma_liste)   # [31, 33, 34, 35]
```

del L[i]

supprime par indice.

remove(x)

supprime par valeur.

Parcours de listes

```
L = [1, 2, 3, 8]
print(len(L))  # 4

ma_liste = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']

# Parcours par indice
i = 0
while i < len(ma_liste):
     print(ma_liste[i])
     i = i + 1

# Parcours direct
for elt in ma_liste:
     print(elt)
```

len(L)

nombre d'éléments

for elt in L

plus lisible

Ajout : l’indentation en Python

```
if age >= 18:
    print("majeur")
    print("accès autorisé")
print("fin")
```

Règle

Après une ligne terminée par :, le bloc suivant doit être indenté.

Conseil

Utiliser toujours 4 espaces par niveau d’indentation.

```
if age >= 18:
print("majeur")   # Erreur : bloc non indenté
```

Ajout : erreurs fréquentes des débutants

Majuscules

True et False ont une majuscule ; if, else, while, for sont en minuscules.

Deux-points

Ne pas oublier ":" après if, elif, else, while, for.

Indentation

Tout bloc doit être indenté de manière cohérente.

Types

input retourne str : convertir avec int ou float si besoin.

Boucles

Ne pas oublier de faire évoluer la variable de contrôle dans une boucle while.

Ajout : exercices rapides

- Écrire un programme qui demande une note et affiche “validé” si la note est ≥ 10.
- Demander un entier n et afficher les nombres pairs de 0 à n.
- Demander 5 valeurs et afficher la plus grande.
- Créer une liste de notes, calculer sa moyenne avec une boucle for.
- Modifier le programme de moyenne pour accepter “oui”, “o”, “non”, “n”.

Les fonctions

La décomposition est une démarche naturelle

+ Une solution à un problème ou la réalisation d'une tâche

+ On identifie les différentes sous-tâches qui composent la résolution du problème ou la réalisation de la tâche

+ Organiser la solution autour de ces sous-tâches

Organiser un voyage pour Paul et Jean:
1 Vérifier les dates de validités des passeports ➔ une fonction de vérification (F1) ➔ F1(Paul) et F1(Jean)
2 Réserver le transport. ➔ une fonction de réservation de transport (F2) ➔ F2(Paul) et F2(Jean)
2 Réserver l'hôtel ➔ une fonction de réservation de l'hôtel ….
3 préparer les bagages ➔ une fonction de préparation de bagage ….
…..

Les fonctions en python

En réalité nous les avons déjà utilisées:

```
A=3
B=2
print(A) # la fonction print
print(A, B)
L=[1,2,3]
len(L) # la fonction len
S=math.sqrt(2.0)
```

+ **Les noms des fonctions :**
  les mêmes règles que pour les noms des variables

Syntaxe

```
def nom_fonction(liste de paramètres):
    instruction1
    instruction2
    instruction3
```

Corps de la fonction:

Comme les instructions if, for et while, l'instruction def est une instruction composée. La ligne contenant cette instruction se termine obligatoirement par un deux-points :, qui introduisent un bloc d'instructions qui est précisé grâce à l'indentation. Ce bloc d'instructions constitue le **corps de la fonction**.

Les fonctions en python

Fonction sans paramètre (sans argument):

```
def compteur3():
    i = 0
    while i < 3:
        print(i)
        i = i + 1
print("bonjour")
compteur3()
compteur3()
```

```
bonjour
0
1
2
0
1
2
```

Une fonction qui appelle une fonction:

```
def double_compteur3():
    compteur3()
    compteur3()

print("bonjour")
double_compteur3()
```

```
bonjour
0
1
2
0
1
2
```

Les fonctions en python

Fonction avec un paramètre

```
def compteur(stop):
    i = 0
    while i < stop:
        print(i)
        i = i + 1
print("bonjour")
compteur(4)
compteur(2)
```

```
bonjour
0
1
2
3
0
1
```

Une fonction qui appelle une fonction:

```
def double_compteur3():
    compteur(3)
    compteur(3)

print("bonjour")
double_compteur3()
```

```
bonjour
0
1
2
0
1
2
```

Les fonctions en python

Fonction avec un paramètre

```
def compteur(stop):
    i = 0
    while i < stop:
        print(i)
        i = i + 1
```

+ L'argument passé à la fonction compteur est la valeur de la variable a.
+ La valeur de a est affectée à la variable stop.
+ Les deux variables a et stop sont physiquement différentes.
+ Même si on remplaçait le nom stop par le nom a, les deux variables seraient distinctes physiquement

Une fonction qui appelle une fonction:

```
def double_compteur3():
    a=3
    compteur(a)
    compteur(a)

print("bonjour")
double_compteur3()
```

```
bonjour
0
1
2
0
1
2
```

Les fonctions avec plusieurs paramètres

Fonction avec plusieurs paramètres

```
def compteur(start, stop, step):
    i = start
    while i < stop:
        print(i)
        i = i + step
A=1
B=7
C=2
Compteur_complet(1,7,2)
Compteur_complet(A,B,C)
```

```
1
3
5
1
3
5
```

+ Pour définir une fonction avec plusieurs paramètres, il suffit d'inclure ceux-ci entre les parenthèses qui suivent le nom de la fonction, en les séparant à l'aide de virgules.

+ Lors de l'appel de la fonction, les arguments utilisés doivent être fournis dans le même ordre que celui des paramètres correspondants (en les séparant eux aussi à l'aide de virgules).

+ Le premier argument sera affecté au premier paramètre, le second argument sera affecté au second paramètre, et ainsi de suite.

+ Encore une fois, les variables données en arguments sans distinctes des variables de la fonctions : les valeurs sont copiées (généralement)

return

Exemple : des fonctions qui s'appellent

Un programme est organisé en plusieurs fonctions. Une fonction peut appeler d'autres fonctions pour réaliser son travail.

main()

```
def main():
    a = lire_nombre("Premier nombre : ")
    b = lire_nombre("Deuxième nombre : ")
    m = maximum(a, b)
    s = moyenne(a, b)
    afficher_resultats(a, b, m, s)
```

lire_nombre(message)

```
def lire_nombre(message):
    n = float(input(message))
    return n
```

Lit un nombre saisi par l'utilisateur et le renvoie.

maximum(x, y)

```
def maximum(x, y):
    if x > y:
        return x
    else:
        return y
```

Calcule et renvoie le plus grand des deux nombres.

moyenne(x, y)

```
def moyenne(x, y):
    s = somme(x, y)
    return s / 2
```

somme(x, y)

```
def somme(x, y):
    return x + y
```

Calcule et renvoie la somme de deux nombres.

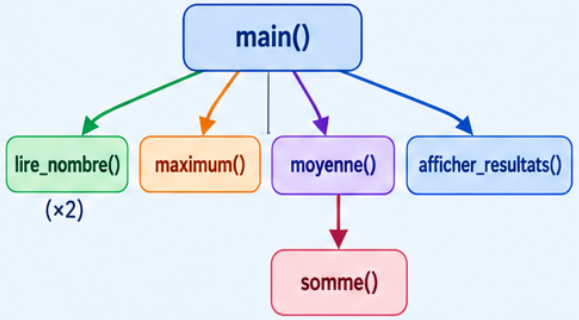
afficher_resultats(a, b, m, s)

```
def afficher_resultats(a,b,m, s):
    print("a =", a)
    print("b =", b)
    print("Le plus grand est :", m)
    print("La moyenne est :", s)
```

Affiche les résultats.

Programmer, c'est organiser une solution en tâches bien identifiées.

Arbre des appels de fonctions



Déroulement des appels

1. main() appelle lire_nombre() (deux fois) pour lire les deux nombres.
2. main() appelle maximum(a, b).
3. main() appelle moyenne(a, b), qui appelle elle-même somme(a, b).
4. main() appelle afficher_resultats(a, b, m, s).

À retenir

- Une fonction peut appeler une ou plusieurs autres fonctions.
- Les appels peuvent s'enchaîner.
- Chaque fonction réalise une tâche précise.
- Cette organisation permet de structurer un programme de manière claire et modulaire.

Exemple: écrire une fonction qui prend en argument un entier et qui dit si il est premier

```
def premier(n):
    if n < 2:
        return False
    d = 2
    while d < n:
        if n % d == 0:
            return False
        d = d + 1
    return True

X=int(input("Donner un entier:")
if(premier(X)):
    print(X,“ est premier“)
else:
    print(X,“ n‘est premier“)
```

```
>> python premier
Donner un entier: 9
9 n’est pas premier
```

```
>> python premier
Donner un entier: 1
1 n’est pas premier
```

```
>> python premier
Donner un entier: 7
7 est pas premier
```

Exemple: écrire une fonction qui prend en argument deux valeurs (bornes) et qui retourne une liste contenant tous les entiers premiers entre les deux bornes

```
def plage_premiers(debut,fin):
    e=debut
    L=[]
    while e <= fin:
        if premier(e):
            L.append(e)
        e+=1
    return L

a=1
b=55
R=plage_premiers(a,b)
print(R)
```

Exemple: dire si un triangle est rectangle

```
def saisir_point():
    x = float(input("x : "))
    y = float(input("y : "))
    return x, y

def distance2(x1, y1, x2, y2):
    return ((x2-x1)**2 + (y2-y1)**2)

def egal(X,Y):
    if abs(X-Y)<0.0001 :
        return True
    else:
        return False

def est_rectangle(a, b, c):
    return egal(a+b,c) or egal(a+c,b) \
           or egal(b+c,a)
```

```
def est_rectangle(XA,YA,XB,YB,XC,YC):
    ab2= distance2(XA, YA, XB, YB)
    ac2= distance2(XA, YA, XC, YC)
    bc2= distance2(XB, YB, XC, YC)

    return est_rectangle(ab2, ac2, bc2)

XA, YA= saisir_point()
XB, YB= saisir_point()
XC, YC= saisir_point()

if est_rectangle(XA,XB,YA,YB,XC,YC):
    print("Oui")
else:
    print(("Non")
```

Exemple: filtrer les nombres non premiers

```
def premier(n):
    if n < 2:
         return False
    d = 2
    while d < n:
        if n % d == 0:
            return False
        d = d + 1
    return True

def filtrer(L):
    R=[]
    i = 0
    while i<len(L):
        if premier(L[i]):
            R.append(L[i])
        i+=1
    return(R)
```

```
def saisir_list():
    R=[]
    n=int(input("Entrez un entier:")
    while True:
        R.append(n)
        rep=input("Voulez vous continuer")
        if(rep=="non"):
            break
        n=int(input("Entrez un entier:")

    return R

H=saisir_list()
R=filtrer(H)
print(R)
```